KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 3
상속과
클래스 변수

틀에 새긴 값과
물려받는 클래스

```
class Bread:
    shop = "붕어빵 1호점"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

오늘의 순서

1	왜 필요한가	복사해서 만든 Dog와 Cat, 틀에 새긴 값과 물려받는 클래스	4–5쪽
2	클래스 변수	틀에 적은 값, 한 번에 바꾸기, 개수 세기, 두 변수 비교	6–9쪽
3	상속	Animal, Dog, Cat으로 물려받기, 더하기, 덮어쓰기, super(), 눌러 보기	10–17쪽
4	복사와 상속 비교	고칠 곳 한 군데, 한 종류일 때만	18–19쪽
5	오류와 함정	super 빠짐, self 두 번, 재료 수, a.shop, self.count	20–24쪽
6	실습	도장 카드와 VIP 카드, 붕어빵과 미니 붕어빵 메뉴판	25–28쪽

3쪽은 노트북 준비, 29–30쪽은 막힐 때와 한 장 요약이에요.

들어가기

25에서 이어서 해요

`class, __init__, self`	24
메서드, a.intro()	24
`__str__, __repr__`	25
지역 변수와 전역 변수	20
for, 컴프리헨션	13, 23

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

개와 고양이 클래스를 따로 만들면 거의 다 복사예요

[1]

```
class Dog:
    def __init__(self, name):
        self.name = name
                              ← 두 클래스가 똑같아요
    def eat(self):
        print(f"{self.name}가 밥을 먹어요")

    def speak(self):   ← 다른 건 여기뿐
        print(f"{self.name}가 멍멍 짖어요")

d = Dog("초코")
d.speak()
```

초코가 멍멍 짖어요

[2]

```
class Cat:
    def __init__(self, name):
        self.name = name
                              ← 두 클래스가 똑같아요
    def eat(self):
        print(f"{self.name}가 밥을 먹어요")

    def speak(self):   ← 다른 건 여기뿐
        print(f"{self.name}가 야옹 울어요")

c = Cat("나비")
c.speak()
```

나비가 야옹 울어요

동물이 늘면 복사도 늘어요

- **class Dog** — __init__, eat 복사
- **class Cat** — __init__, eat 복사
- **class Rabbit** — __init__, eat 복사
- **class Duck** — __init__, eat 복사
- **class Cow** — __init__, eat 복사

1. 왜 필요한가

틀에 새기는 값, 그리고 물려받는 클래스

틀 class Bread

```
shop = "붕어빵 1호점"   ← 클래스 변수
__init__(flavor, price)
intro()
```



a	Bread
flavor	"팥"
price	1000

b	Bread
flavor	"피자"
price	1500

a도 b도 틀의 shop을 같이 봐요

클래스 class Animal

```
__init__(name), eat(), speak()
```

물려받기 = 상속

클래스 class Dog(Animal)

```
speak()  멍멍
```

클래스 class Cat(Animal)

```
speak()  야옹
```

Dog와 Cat에는 speak만 적어요.
__init__과 eat는 Animal에게서 물려받아요.

2. 클래스 변수

틀에 바로 적은 값은 모든 붕어빵이 같이 봐요

[1]
```
class Bread:
    shop = "붕어빵 1호점"    ← 틀에 바로 적기

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.shop)
print(b.shop)
print(Bread.shop)
```

```
붕어빵 1호점
붕어빵 1호점
붕어빵 1호점
```
셋 다 틀의 값



① a 안에서 shop 찾기 ✕ 없어요

② a를 찍은 틀 Bread에서 찾기 ✓ 붕어빵 1호점

2. 클래스 변수

틀의 값을 바꾸면 모든 붕어빵에 한 번에 바뀌어요

```
[2] Bread.shop = "붕어빵 2호점"    ← 틀 이름으로 바꿔요
    print(a.shop)
    print(b.shop)
```

```
붕어빵 2호점
붕어빵 2호점
```

틀 `class Bread`

`shop = "붕어빵 2호점"`



붕어빵이 100개여도 Bread.shop 한 줄이면 돼요.

2. 클래스 변수

찍을 때마다 틀에 개수를 세요

[3]
```
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        Bread.count = Bread.count + 1 ← 틀에 +1

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
c = Bread("피자", 1500)
print(Bread.count)
```

```
3
```

Bread.count

3

`a = Bread("팥", 1000)`	+1 → 1
`b = Bread("슈크림", 1200)`	+1 → 2
`c = Bread("피자", 1500)`	+1 → 3

가게 이름, 전체 개수처럼 틀 전체에 하나만 있으면 되는 값에 써요.

2. 클래스 변수

클래스 변수와 인스턴스 변수

	인스턴스 변수	클래스 변수
어디에 적나요	`__init__` 안, `self.flavor = ...`	`class` 바로 아래, `shop = ...`
누구 것인가요	붕어빵 하나하나 (객체)	틀 하나 (클래스)
몇 개 있나요	붕어빵 수만큼	틀에 하나
꺼내기	`a.flavor`	`a.shop`, `Bread.shop`
바꾸기	`a.flavor = ...`	`Bread.shop = ...`
예	맛, 가격, 도장 수	가게 이름, 전체 개수

인스턴스는 객체와 같은 말이에요. 붕어빵마다 다르면 self.이름, 모두 같으면 틀에 적어요.

3. 상속

괄호에 부모 클래스를 적으면 전부 물려받아요

[1]
```
class Animal:
    def __init__(self, name):
        self.name = name
                                ← 모두 같이 쓰는 것
    def eat(self):
        print(f"{self.name}가 밥을 먹어요")

    def speak(self):
        print(f"{self.name}가 소리를 내요")
```

[2]
```
class Dog(Animal):  ← 괄호에 부모 클래스
    pass

d = Dog("초코")
d.eat()
d.speak()
```

```
초코가 밥을 먹어요
초코가 소리를 내요
```
pass뿐인데 둘 다 돼요



3. 상속

자식 클래스에는 더 할 일을 적어요

[3]
```
class Dog(Animal):
    def fetch(self):
        print(f"{self.name}가 공을 물어 와요")

a = Animal("하마")
d = Dog("초코")
d.eat()
d.fetch()
```

Dog에만 있는 메서드

```
초코가 밥을 먹어요
초코가 공을 물어 와요
```

← eat는 부모에게서, fetch는 자기 클래스에서

[4]
```
a.fetch()
```

```
AttributeError: 'Animal' object has no attribute 'fetch'
```

← a는 Animal로 만들었어요

물려받기는 부모에서 자식 쪽으로만 일어나요. 모든 동물이 공을 물어 오지는 않아요.

3. 상속

같은 이름의 메서드를 다시 적으면 자식 것이 쓰여요

[5]
```
class Dog(Animal):
    def speak(self):   ← 부모에도 있는 이름
        print(f"{self.name}가 멍멍 짖어요")

d = Dog("초코")
d.speak()
```

초코가 멍멍 짖어요 ← Dog의 speak

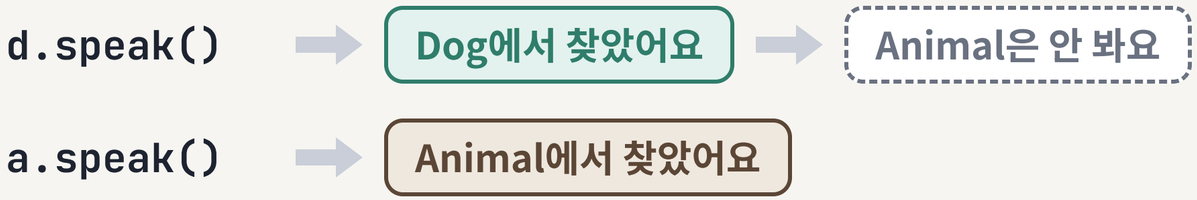
[6]
```
class Cat(Animal):
    def speak(self):   ← 부모에도 있는 이름
        print(f"{self.name}가 야옹 울어요")

c = Cat("나비")
c.speak()
```

나비가 야옹 울어요 ← Cat의 speak

[7]
```
a.speak()
```

하마가 소리를 내요 ← Animal의 speak 그대로



메서드는 자식 클래스에서 먼저 찾고, 없으면 부모 클래스로 올라가요.

3. 상속

super()로 부모 것을 부르고 조금만 더할 수도 있어요

[8]
```
class Dog(Animal):
    def eat(self):
        super().eat()                                  ← super()는 부모 클래스. 부모의 eat를 그대로 불러요
        print(f"{self.name}가 꼬리를 흔들어요")          ← 한 줄만 더해요


d = Dog("초코")
d.eat()
```

```
초코가 밥을 먹어요          ← Animal의 eat가 찍은 줄
초코가 꼬리를 흔들어요      ← Dog가 더한 줄
```

eat	`super().eat()`	부모의 eat 부르기
__init__	`super().__init__(name)`	부모의 `__init__` 부르기 (다음 장)

3. 상속

칸을 더할 땐 super().__init__으로 부모에게 맡겨요

[9]
```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed  ← 개만 가진 칸은 직접 붙여요

    def speak(self):  ← 앞에서 본 멍멍
        print(f"{self.name}가 멍멍 짖어요")

d = Dog("초코", "푸들")
print(d.name, d.breed)
d.speak()
```

```
초코 푸들
초코가 멍멍 짖어요
```

`super()` — 부모 클래스

`.__init__(name)` — 부모의 __init__에 name을 넣어요. self는 안 적어요



name은 부모가, breed는 자식이 붙여요.

3. 상속

Dog("초코", "푸들") 한 줄에서 일어나는 일

① **`Dog.__init__`**
자식의 __init__이 불려요. self는 새 객체

② **`super().__init__(name)`**
부모의 __init__으로 가요. 같은 self예요

③ **`self.name = name`**
부모가 name 칸을 붙여요

④ **`self.breed = breed`**
자식으로 돌아와 breed 칸을 붙여요

⑤ **`d = 다 만든 객체`**
두 칸이 다 붙은 객체가 d에 담겨요



부모의 __init__에서도 자식의 __init__에서도 같은 self예요

3. 상속

섞어 담아도 동물마다 자기 speak를 불러요

[10]
```
animals = [Dog("초코", "푸들"), Cat("나비"), Animal("하마")]
for x in animals:
    x.speak()  ← 부르는 줄은 하나
```

```
초코가 멍멍 짖어요     ← Dog의 speak
나비가 야옹 울어요     ← Cat의 speak
하마가 소리를 내요     ← Animal의 speak
```

[11]
```
print(isinstance(d, Dog))      ← isinstance(값, 클래스): 그 클래스로 만들었나요?
print(isinstance(d, Animal))
print(isinstance(a, Dog))
```

```
True    ← d는 Dog로 만들었어요
True    ← 개도 동물이에요
False   ← 하마는 개가 아니에요
```

3. 상속

눌러 보기: 파이썬은 어디서부터 찾을까요

클래스 `class Animal`

```
__init__(self, name)
eat(self)
speak(self)  # 소리를 내요
```

물려받기

클래스 `class Dog(Animal)`

```
__init__(self, name, breed)
speak(self)  # 멍멍
fetch(self)
```

a — Animal

`name "하마"`

Animal로 만들었어요

d — Dog

`name "초코"`

`breed "푸들"`

Dog로 만들었어요

누구에게 [a] [d]

무엇을 [.speak()] [.eat()] [.fetch()] [.breed] [.name]

찾아보기 ▶ `d.eat()`

찾는 길이 여기에 한 줄씩 나와요.

출력

누구에게 무엇을 물을지 고르고 찾아보기를 눌러요. 브라우저로 슬라이드를 열면 눌러져요.

4. 복사와 상속 비교

복사해서 만든 클래스와 물려받은 클래스

복사 (1파트)

```
class Cat:
    def __init__(self, name):
        self.name = name
                                ← Dog에도 똑같이 적음
    def eat(self):
        print(f"{self.name}가 밥을 먹어요")

    def speak(self):
        print(f"{self.name}가 야옹 울어요")
```

상속 (3파트)

```
class Cat(Animal):
    def speak(self):
        print(f"{self.name}가 야옹 울어요")
```

__init__과 eat는 Animal에게 맡겼어요.
다른 점인 speak만 적어요.

	복사	상속
eat를 고치면	Dog, Cat 다 고쳐야 해요	Animal 한 곳만 고쳐요
적는 것	모든 줄	다른 점만

4. 복사와 상속 비교

"~는 ~의 한 종류"일 때 상속해요

✓ 상속	개는 동물의 한 종류예요	`Dog(Animal)`
✓ 상속	고양이는 동물의 한 종류예요	`Cat(Animal)`
✓ 상속	미니 붕어빵은 붕어빵의 한 종류예요	`MiniBread(Bread)`
× 따로	동물원은 동물의 한 종류가 아니에요	Zoo 클래스를 따로
× 따로	사료는 동물의 한 종류가 아니에요	Food 클래스를 따로

코드가 비슷해 보여도 한 종류가 아니면 상속하지 않아요.

5. 오류와 함정

super().__init__을 빼먹으면 부모 칸이 없어요

[1]
```
class Animal:
    def __init__(self, name):
        self.name = name

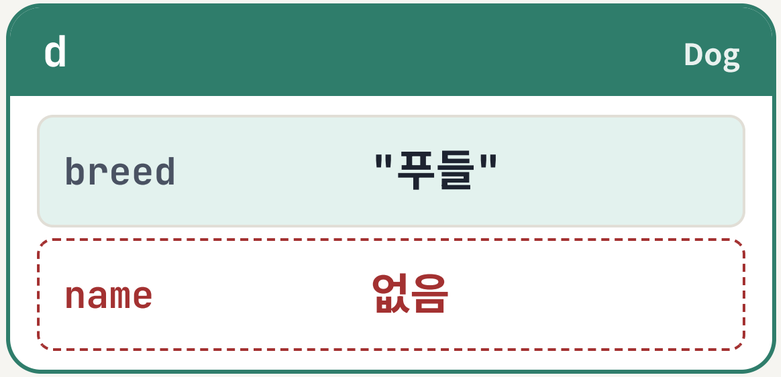
    def eat(self):
        print(f"{self.name}가 밥을 먹어요")

    def speak(self):
        print(f"{self.name}가 소리를 내요")

class Dog(Animal):
    def __init__(self, name, breed):
        self.breed = breed  ← super().__init__ 줄이 없어요

d = Dog("초코", "푸들")
d.eat()
```

```
AttributeError: 'Dog' object has no attribute 'name'
```



자식 __init__ 첫 줄에 적어요

✓ `super().__init__(name)`

5. 오류와 함정

super().__init__에 self를 넣으면 3 were given

[2]
```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(self, name)  ← self를 직접 넣었어요
        self.breed = breed

d = Dog("초코", "푸들")
```

```
TypeError: Animal.__init__() takes 2 positional arguments but 3 were given
```

Animal.__init__의 칸 — self | name

들어온 값 3개 — self (자동) | self | name

✓ `super().__init__(name)`

5. 오류와 함정

자식 클래스는 재료가 하나 더 필요해요

[3]
```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

d = Dog("초코")  ← 이름만 넣었어요
```

```
TypeError: Dog.__init__() missing 1 required positional argument: 'breed'
```



✕ ~~Dog("초코")~~ → ✓ `Dog("초코", "푸들")`

견종을 자주 안 적으면 20에서 배운 기본값으로 breed="믹스"를 둘 수도 있어요.

a.shop = 으로 바꾸면 a에만 새 칸이 생겨요

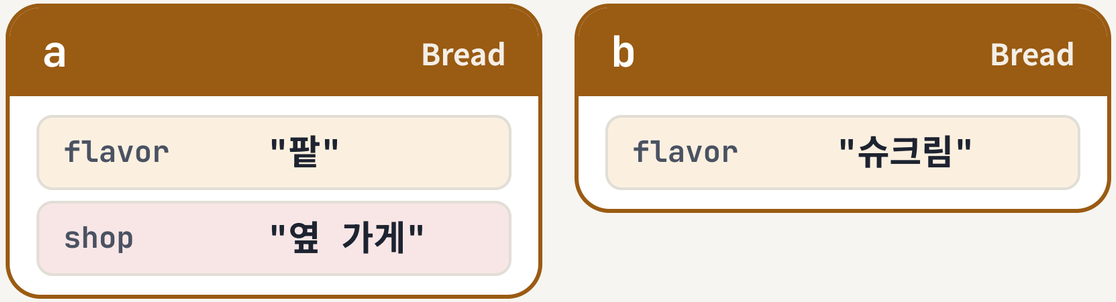
[4]
```
class Bread:
    shop = "붕어빵 1호점"

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.shop = "옆 가게"   ← 붕어빵 이름으로 바꾸기
print(a.shop)
print(b.shop)
print(Bread.shop)
```

옆 가게 ← a에 붙은 칸이 먼저 보여요
붕어빵 1호점
붕어빵 1호점] b와 틀은 그대로

틀 class Bread
shop = "붕어빵 1호점" ← 그대로예요



모두 바꾸려면 틀 이름으로

× ~~a.shop = "옆 가게"~~

✓ Bread.shop = "옆 가게"

self.count로 세면 틀의 개수가 안 늘어요

[5]
```
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        self.count = self.count + 1

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.count, b.count)
print(Bread.count)
```

```
1 1    ← a.count, b.count
0      ← Bread.count
```

✓ `Bread.count = Bread.count + 1`

`self.count = self.count + 1`

왼쪽: 붕어빵에 새 칸을 붙여요

오른쪽: 붕어빵에 없으니 틀의 0을 읽어요

틀 class Bread

count = 0 ← 한 번도 안 바뀌어요

a	Bread
flavor	"팥"
count	1

b	Bread
flavor	"슈크림"
count	1

6. 실습

도장 카드 ①: 틀에 가게와 목표, VIP는 두 배로

[1]

```
class Card:
    shop = "KANT 카페"   ← 모든 카드가 같은 값: 클래스 변수
    goal = 10

    def __init__(self, name):
        self.name = name
        self.stamps = 0

    def add(self):
        self.stamps = self.stamps + 1

    def __str__(self):   ← 남은 도장 수까지 글자로 (25)
        left = Card.goal - self.stamps
        return f"[{Card.shop}] {self.name}: 도장 {self.stamps}개, {left}개 남음"

class VipCard(Card):   ← 도장 카드의 한 종류, add만 두 개씩
    def add(self):
        self.stamps = self.stamps + 2
```

출력 없이 틀만 만드는 셀이에요

6. 실습

도장 카드 ②: 같은 add인데 카드마다 다르게

```
[2] kim = Card("kim")
    lee = VipCard("lee")
    kim.add()
    kim.add()
    lee.add()
    lee.add()
    print(kim)
    print(lee)
```

```
[KANT 카페] kim: 도장 2개, 8개 남음
[KANT 카페] lee: 도장 4개, 6개 남음
```

kim — Card

name	"kim"
stamps	2

lee — VipCard

name	"lee"
stamps	4

`kim.add()` → `Card.add` +1씩

`lee.add()` → `VipCard.add` +2씩

print와 가게 이름은 둘 다 Card에서 물려받아요

6. 실습

붕어빵 메뉴판 ①: 미니 붕어빵 틀을 물려받기

[3]

```
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        Bread.count = Bread.count + 1  ← 찍을 때마다 개수 세기

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

class MiniBread(Bread):  ← 미니 붕어빵도 붕어빵의 한 종류
    def __init__(self, flavor, price, qty):
        super().__init__(flavor, price)  ← 맛과 가격은 부모에게 맡겨요
        self.qty = qty

    def __str__(self):  ← 묶음 가격이 보이게 덮어쓰기
        return f"미니 {self.flavor} {self.qty}개 {self.price * self.qty:,}원"
```

이 셀도 출력이 없어요

6. 실습

붕어빵 메뉴판 ②: 섞어 담고 찍고 세기

[4]
```
menu = [Bread("팥", 1000), MiniBread("팥", 300, 5), Bread("슈크림", 1200), MiniBread("슈크림", 400, 5)]
for m in menu:
    print(m)
print("찍은 붕어빵:", Bread.count)
print([m.flavor for m in menu if isinstance(m, MiniBread)])
```

```
팥 붕어빵 1,000원
미니 팥 5개 1,500원
슈크림 붕어빵 1,200원
미니 슈크림 5개 2,000원
찍은 붕어빵: 4
['팥', '슈크림']
```

붕어빵마다 자기 틀의 __str__ (25)

← 미니도 부모 __init__을 거쳐서 4개

← isinstance와 컴프리헨션(23)으로 미니만

24부터 26까지 배운 클래스를 한 셀에서 다 썼어요.

정리

막히면 오류의 마지막 줄을 봐요

1	`AttributeError: 'Dog' object has no attribute 'name'` 자식 __init__에서 super().__init__을 안 불렀어요	`super().__init__(...)`
2	`TypeError: ... takes 2 positional arguments but 3 were given` super().__init__에 self를 넣었어요	self 빼기
3	`TypeError: Dog.__init__() missing ... 'breed'` 자식 클래스의 재료가 모자라요	칸 수 맞추기, 기본값
4	`NameError: name 'Anmal' is not defined` 괄호 안 부모 이름 오타이거나 부모를 아직 안 만들었어요	부모 셀 먼저 실행
5	**한 붕어빵만 값이 달라요** a.shop = 으로 바꿨어요	`Bread.shop =`
6	**개수가 0에서 안 늘어요** self.count로 셌어요	`Bread.count`
7	**부모 메서드를 고쳤는데 자식은 그대로예요** 자식이 같은 이름으로 덮어썼어요	super().메서드()

헷갈리면 "자식 클래스에서 먼저 찾고, 없으면 부모 클래스" 순서로 따라가 봐요.

한 장 요약

`shop = "1호점"`
틀에 새긴 값 (클래스 변수)

`Bread.shop = ...`
모두 한 번에 바꾸기

`class Dog(Animal):`
클래스 물려받기

`super().__init__(name)`
칸을 더할 때 부모에게 맡기기

`def speak(self):`
같은 이름이면 덮어쓰기

`super().eat()`
부모 것을 부르고 조금 더

`isinstance(d, Animal)`
어떤 클래스로 만들었나

~는 ~의 한 종류
상속할지 고르는 말

객체마다 다르면 self.이름, 모두 같으면 클래스에 적어요. 한 종류면 물려받아요.

클래스 세 챕터 끝: 클래스로 데이터와 할 일을 한 곳에 묶었어요.